

Polymarket

量化交易系统诊断报告

V3.5 系统全面审计

审计日期: 2026-05-17

审计范围: 17个源码文件, 约4500行Python代码

执行摘要

经过对 Polymarket 量化交易系统 V3.5 全部17个源码文件（约4500行Python代码）的深度审计，发现3个致命缺陷（P0）、5个严重缺陷（P1）和5个中等缺陷（P2）。其中最关键的问题是Kelly仓位计算存在双重置信度惩罚导致系统几乎不可能下单，信号概率估计为硬编码加5%使Kelly公式输入无效，以及校准引擎的实际结果定义错误导致反馈学习方向相反。

等级	数量	核心问题	影响
P0 致命	3	Kelly死锁/信号硬编码/校准错误	系统无法盈利
P1 严重	5	假钱包/双止损冲突/套利无机会/仓位混乱/条件过严	大幅降低有效性
P2 中等	5	SQLite并发/单线程HTTP/WS不稳定/死代码/前端过载	影响健壮性

结论：当前系统处于"看起来很专业但实际无法交易"的状态。12步闭环数据流的架构设计是正确的，但核心计算模块的实现存在根本性错误，导致整个闭环空转。需要立即修复P0问题后才能进行有效交易。

P0 致命缺陷

缺陷1: Kelly=0 死锁 — 系统永远不会下注

在 kelly_criterion.py 的 confidenceAdjustedKelly 函数中，当 sample_size=0（新系统必然如此）时，shrinkage 被设为 confidence 的值，而 adjusted_f 的计算公式为 kelly_fraction 乘以 confidence 再乘以 shrinkage，由于 shrinkage 等于 confidence，这就形成了 confidence 被乘了两次的双重惩罚。

具体代码路径（kelly_criterion.py 第228-232行）：当 sample_size == 0 时，shrinkage = confidence; 然后 adjusted_f = kelly_fraction * confidence * shrinkage。由于 shrinkage 已经等于 confidence, 实际计算为 adjusted_f = kelly_fraction * confidence * confidence, 即 confidence 被平方。

数值推演: 假设 edge = 5%, price = 0.50, 则 raw kelly = 0.05/0.50 = 0.10; confidence = min(0.05*5, 1.0) = 0.25; adjusted_f = 0.10 * 0.25 * 0.25 = 0.00625 (0.625%); position_size = \$100 * 0.00625 = \$0.625, 远低于 MIN_TRADE_SIZE = \$5, 交易被拒绝。

即使将 edge 提高到 10%, kelly_fraction = 0.20, confidence = 0.50, 调整后 adjusted_f = 0.20 * 0.50 * 0.50 = 0.05 (5%), position_size = \$5.0, 刚好触底。Kelly仍然极低，系统几乎不

可能产生有效仓位。

修复方案: 将双重惩罚改为单次惩罚。当 `sample_size == 0` 时, 使用 `shrinkage = min(1.0, confidence * 2)` 给新系统更多空间; `adjusted_f = kelly_fraction * shrinkage`, 不再乘以 `confidence`; 然后应用 `kelly_cap` 上限即可。

缺陷2: 信号概率估计硬编码"加5%"

在 `server.py` 第1694-1697行的 `process_trade_opportunity` 函数中, 策略信号概率的估算方式为: 当 `side == YES` 时, `strategy_prob = min(0.95, trade_price + 0.05)`; 当 `side == NO` 时, `strategy_prob = max(0.05, 1 - trade_price - 0.05)`。这意味着每种策略都假设固定5%的边际优势, 完全无视策略逻辑的差异。

例如, 均值回归策略在 `YES=0.10` 的极端低价时, 概率估计仅为0.15, 但真正高潜力的均值回归应该给出0.25-0.35的概率估计。事件驱动策略在剧烈波动时, 概率仍然只加了5%。这种固定加成让Kelly公式的核心输入变成垃圾输入, 导致垃圾输出。

修复方案: 为每种策略定制概率估计函数。均值回归应使用 `trade_price + (0.50 - trade_price) * 0.4`, 让极端价格获得更大的反转概率; 事件驱动应使用 `price_change` 幅度乘以系数; 0手续费策略应考虑免费交易带来的额外边际。

缺陷3: 校准引擎"实际结果"定义错误

在 `server.py` 第1657行, 校准引擎记录实际结果时使用了: `actual_outcome = 1 if pos.pnl > 0 else 0`。这意味着用"是否盈利"来替代"事件是否真的发生了"。Brier Score 校准要求 `actual_outcome` 代表事件的真实发生情况 (1=发生了, 0=没发生), 而用盈利与否替代会导致校准引擎学到完全错误的反馈。

一个 `YES=0.90` 的市场, 你以0.91买入, 结果价格跌到0.89亏钱了, 但事件实际上可能还是发生了。这个错误会导致Step 12的策略权重调整方向与实际相反, 使得好的策略被降权, 差的策略被增权。

修复方案: 短期替代方案是使用持仓方向与价格变动方向判断: `actual_outcome = 1 if (pos.side == YES and pos.current_price > pos.entry_price) or (pos.side == NO and pos.current_price < pos.entry_price) else 0`。长期方案是在市场结算时记录真实结果。

P1 严重缺陷

缺陷4: Smart Money 钱包地址是伪造的

smart_money_tracker.py 第23-48行中的三个 KNOWN_SMART_WALLETS 地址是随机编造的42位十六进制字符串, 不是真实的 Polymarket 高盈利钱包。Gamma API 对这些地址会返回空数据, 导致 Smart Money 模块形同虚设, 永远不会产生有效信号。

修复方案: 替换为 Polymarket 链上可验证的真实盈利钱包地址, 可通过 Polymarket 排行榜或 Polygonscan 链上分析获取。

缺陷5: 双止损系统冲突

risk_manager.py 使用固定3%止损, risk_manager_v3.py 使用ATR动态止损。两个系统同时运行: 当动态止损认为"还没到"时, 基本3%止损可能已经触发强制平仓。3%止损对预测市场来说太紧, 一个YES=0.10的仓位, 价格从0.10跌到0.097就触发止损, 这几乎是市场噪音。

修复方案: 将基本止损从3%放宽到8%, 止盈从8%放宽到15%; 当动态止损有足够ATR数据时, 优先使用动态止损, 基本止损仅作为兜底。

缺陷6: 套利策略几乎无机会

单市场YES+NO套利在代码注释中已写明"实测几乎不存在"。扣除taker fee后, YES+NO总价格几乎不可能低于1.0。多市场套利需要negRisk事件的YES总和偏离1.0超过2%加手续费, 这种情况极为罕见。这两个策略消耗扫描时间但不产生收益。

修复方案: 降低套利权重, 将扫描时间优先分配给0手续费和均值回归策略; 或开发跨交易所套利。

缺陷7: 仓位计算体系混乱

三套仓位计算互相矛盾: risk_manager.py 的 calculate_position_size 使用固定8%加上限15%; kelly_criterion.py 的 calculate_position_size_kelly 使用Kelly分数加上限15%; server.py 套利分支直接调用 calculate_position_size, 绕过了Kelly。不同策略使用不同的仓位计算方式, 导致风控行为不一致。

缺陷8: 事件驱动策略条件过严

事件驱动策略必须满足 is_extreme_price (YES小于0.10或YES大于0.90) 才交易。极端价格市场流动性差、价差大、手续费侵蚀严重, 而且这些价格往往是"正确的" (事件确实不太可能发生)。这导致该策略几乎不会触发, 即使触发也容易亏损。

P2 中等缺陷

编号	缺陷	位置	影响	修复方案
----	----	----	----	------

9	SQLite并发风险	data_store.py	可能"database is locked"	启用WAL模式
10	HTTP Server单线程	server.py	14个API串行处理	迁移到异步框架
11	WebSocket依赖不稳定	orderbook_engine.py	降级20s轮询延迟	安装websocket-client
12	bot.py死代码	bot.py	混淆维护	删除或标记废弃
13	前端14路并行刷新	server.py HTML	5秒14请求过载	合并API+缓存

修复优先级与方案

优先级	修复项	涉及文件	预估工作量
1	修复Kelly双重惩罚	kelly_criterion.py	10行代码
2	信号概率策略化	server.py	30行代码
3	校准引擎结果定义	server.py	5行代码
4	放宽止损参数	config.py	2行配置
5	替换真实钱包地址	smart_money_tracker.py	3行数据
6	统一仓位计算体系	server.py + risk_manager.py	20行代码
7	SQLite WAL模式	data_store.py	2行代码

修复P0的3个问题后, 系统即可开始产生有效交易。P1问题建议在第一周内修复, P2问题可在后续迭代中优化。建议修复顺序严格按照上表优先级执行, 先让系统能够正常交易, 再优化交易质量。